

API Handling Notes - POST Request & Environment Variables

1. What is POST?

POST sends data from the client to the server. Common uses: Login, Registration, Create Employee, Place Order, Upload File.

Memory Trick: GET = Get Data | POST = Push/Send Data.

2. GET vs POST

GET retrieves data using URL parameters. POST sends data in the request body and usually creates or updates resources.

3. Structure of a POST Request

1. URL
2. Headers
3. Body
4. Query Parameters (Optional)

4. Request Body

Contains the actual data being sent to the server. Example payload:
{'name':'Koustav','city':'Kolkata'}

5. Headers

Provide metadata about the request. Common headers: Content-Type, Authorization, Accept.

6. Sending JSON

Use `requests.post(url, json=payload)`. It automatically converts a Python dictionary to JSON and sets Content-Type: application/json.

7. Form Data

Used by HTML forms and some legacy APIs. Example:
`username=koustav&password;=123456`

8. Sending Form Data

Use `requests.post(url, data=payload)`. Default Content-Type becomes application/x-www-form-urlencoded.

9. json= vs data=

`json=` -> JSON -> application/json. `data=` -> Form Data -> application/x-www-form-urlencoded.

10. Authorization Header

Used to authenticate API requests. Example: Authorization: Bearer .

11. Environment Variables

Store secrets outside your code, e.g. API_TOKEN and BASE_URL.

12. .env File

Create a .env file in the project root. Example:

```
API_TOKEN=my_secret_token  
BASE_URL=https://httpbin.org
```

13. python-dotenv

Install using: `pip install python-dotenv`

14. Load .env

```
from dotenv import load_dotenv  
load_dotenv()
```

15. Read Variables

```
import os  
os.getenv('API_TOKEN')  
os.getenv('BASE_URL')
```

16. Best Practices

Never hardcode secrets. Never commit .env to Git. Add .env to .gitignore.

Quick Revision

GET = Retrieve
POST = Send
json= -> JSON
data= -> Form Data
Authorization -> Bearer Token
Secrets -> .env
Read using `load_dotenv()` and `os.getenv()`.